

CASA AL1 — Evidence Pack, Round 2

Application: Sonave (usesonave.com) · **Assessment:** TAC Security ESOF, CASA Tier 2 (AL1) **Date of evidence collection:** 2026-09-04 (all command outputs below captured live on this date) **Scope of this pack:** The 15 requirements marked *Fail — evidence insufficient* in the assessor's first verification round. No finding in that round reported an exploitable issue; each item below supplies the specific artifact the tester comment requested (source code with file:line references, live scan output, DNS records, or log samples). Source references are to the production repository deployed to Railway (branch `main`).

Row 29 — 4.1.3 · No weak cryptography

Tester asked for: documentation of cryptographic operations, key sizes, key generation, IV handling, and key management.

Cryptographic inventory (complete — every crypto operation in the application layer):

Operation	Algorithm	Location
Session token signing/verification	HMAC-SHA256	<code>railway/auth.py:94-111</code>
Machine/operator token comparison	constant-time <code>secrets.compare_digest</code>	<code>railway/app.py:101</code> , <code>railway/auth.py:307</code>
Bot token storage (at rest)	SHA-256 hash only, plaintext never stored	<code>railway/app.py:372</code> , <code>lookup app.py:542</code>
API key storage (at rest)	SHA-256 hash only	<code>railway/db.py:604</code>
Outbound webhook signing	HMAC-SHA256 (<code>t={ts},v1={sig}</code> scheme)	<code>railway/webhook_dispatcher.py:205-226</code>
Stripe webhook verification	HMAC-SHA256 + <code>compare_digest</code>	<code>railway/billing.py:61-72</code>
Forensic report / compliance vault integrity seals	SHA-256 digest + HMAC-SHA256 signature	<code>railway/forensics.py:19-31</code> , <code>railway/compliance_vault.py:44-72</code>

Key generation and sizes: all secret material is generated by Python's CSPRNG `secrets` module — session/bot/API tokens via `secrets.token_urlsafe(32)` = 256 bits of entropy (`railway/app.py:349` , `railway/db.py:609` , `railway/auth.py:133`). HMAC signing keys are 256-bit values supplied via environment variables (see Row 48).

Signing implementation (verbatim, `railway/auth.py:94-111`):

```
def _sign(payload: dict) -> str:
    body = _b64e(json.dumps(payload, separators=(",", ":")).encode())
    sig = _b64e(hmac.new(_session_secret(), body.encode(), hashlib.sha256).digest())
    return f"{body}.{sig}"

def _verify(token: str) -> dict | None:
    try:
        body, sig = token.split(".", 1)
        want = _b64e(hmac.new(_session_secret(), body.encode(), hashlib.sha256).digest())
        if not hmac.compare_digest(sig, want):
            return None
        payload = json.loads(_b64d(body))
        if payload.get("exp", 0) < time.time():
            return None
        return payload
    except Exception: # any malformed token is just invalid
        return None
```

IV handling: not applicable — the application performs **no application-layer symmetric encryption** (no block/stream cipher usage, therefore no IVs or nonces to manage). Confidentiality in transit is provided by TLS 1.3 at the Cloudflare edge (AEAD suites: AES-256-GCM / ChaCha20-Poly1305 — see passed rows 4.1.1/4.1.2); confidentiality at rest by the hosting platform's encrypted volumes (Row 48).

No deprecated algorithms: repository-wide case-insensitive search for `md5` and `sha1` across all Python sources in the deployed service returns **zero matches** (search performed 2026-09-04; the only hash algorithms present are SHA-256 and HMAC-SHA256, as listed above).

Key management: keys are injected at runtime from Railway environment variables (never in source or VCS), rotation and revocation procedures are documented in `docs/secrets-policy.md` (provided with Row 48). Per-user session revocation is immediate via the server-side `session_ver` counter (`bump_session_ver` , verified under passed rows 2.2.1/2.2.2).

Row 30 — 4.1.4 · Cryptographic modules fail securely / no padding oracle

Tester asked for: implementation details showing cryptographic failures return generic, consistent errors with no information usable for padding-oracle or side-channel attacks.

1. **No padding oracle surface exists.** The application performs no decryption of any kind — no CBC mode, no ciphertext parsing, no application-layer encryption at all (see Row 29). Padding oracles require a decryption operation whose error behavior distinguishes padding validity; there is no such operation in this codebase.
2. **All MAC/token verification is constant-time and collapses every failure mode into one result.** `_verify` (verbatim above, `railway/auth.py:100-111`) returns `None` identically for: bad signature (checked with `hmac.compare_digest` , line 104), expired token, malformed base64, malformed JSON, or any other exception (bare `except` at line 110). The caller cannot distinguish *why* a token failed.
3. **Uniform error responses.** Every authentication failure surfaces as the same generic HTTP 401 (`{"detail":"unauthorized"}`) or 403; no cryptographic detail, stack trace, or distinguishing message is ever returned (default-deny behavior verified live by the assessor under passed row 3.1.3).
4. **Constant-time comparison everywhere secrets are compared:** `railway/app.py:101` , `railway/auth.py:104` , `railway/auth.py:143` , `railway/auth.py:307` , `railway/billing.py:72` , `railway/webhook_dispatcher.py:226` , `railway/app.py:1413` .

Row 34 — 5.1.4 · Template injection

Tester asked for: source/template-configuration evidence that user input cannot reach template logic.

The application uses no template engine. Pages are static HTML files shipped in the repository and served with literal string replacement of a fixed set of server-controlled placeholder tokens — never user input. Verbatim serving code (`railway/app.py:2196-2198` , representative of all page routes at `app.py:2173-2318`):

```
content = (html.replace("__FAVICON__", _FAVICON_B64)
            .replace("__MEET_PROJECT__", os.environ.get("SONAVE_MEET_PROJECT_NUMBER", ""))
            .replace("__GOOGLE_CID__", os.environ.get("SONAVE_GOOGLE_CLIENT_ID", "")))
```

- Replacement values are exclusively **server environment constants** (favicon bytes, Google project number, OAuth client ID). No request parameter, header, or body value is ever interpolated into HTML server-side.
- Repository-wide search confirms **zero imports or usage** of Jinja2, Mako, string.Template, or any other template engine in application code (`jinja2` appears in the dependency tree only as a transitive dependency of the ML stack and is never imported by the service — searched 2026-09-04).
- Because no template is ever *evaluated*, server-side template injection is structurally impossible; there is no expression evaluator for attacker input to reach.

Row 37 — 5.1.7 · Context-aware output escaping (XSS)

Tester asked for: evidence that context-aware output escaping/sanitization is applied to application outputs, beyond the CSP already verified live.

Server side: all API endpoints return `application/json` (FastAPI serialization — auto-escaped by content type); HTML routes serve static templates only (Row 34), so no user-controlled data is emitted in HTML by the server.

Client side: every dynamic render in the two applications passes untrusted data through a single HTML-entity encoder before it reaches an `innerHTML` sink. The encoder (`railway/console.html:675`):

```
function esc(s){return String(s==null?'':s).replace(/[\<>"]/g,
  c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;','"':'&#39;'}[c]))}
```

Applied at every sink that renders untrusted data — speaker names (attacker-influenceable via meeting display names), emails, model IDs, filenames, incident details: `console.html:736, 903-905, 918-921, 1026, 1104-1105, 1115-1119, 1188, 1272, 1377, 1640, 1662, 1693-1700, 1712-1713`; identically in the Meet add-on (`meet-addon.html:479, 486` , list rows via `esc()` in `rowHTML`). The remaining `innerHTML` assignments compose only server-side constants and application-defined markup (nav icons, static hint strings), with no untrusted operand.

Defense in depth (verified live on 2026-09-04): production responses carry `Content-Security-Policy` with `object-src 'none'; base-uri 'self'; form-action` restrictions, plus `X-Content-Type-Options: nosniff` and `Referrer-Policy: strict-origin-when-cross-origin` (header capture in Row 43 below applies to all routes).

Row 38 — 5.1.8 · Database injection

Tester asked for: evidence of the database access implementation (parameterized queries / allowlisting).

All database access goes through `railway/db.py` using SQLite **parameterized ? bindings exclusively** — user-controlled values are never concatenated into SQL text. The one function that builds column names dynamically is guarded by a frozen allowlist (verbatim, `railway/db.py:270-282`):

```
def mark_bot(bot_id: str, **cols) -> None:
    if not cols:
        return
    # Whitelist allowed column names to prevent SQL injection via f-string (OWASP A03)
    _ALLOWED_BOT_COLS = frozenset({
        "status", "started_ts", "ended_ts", "metered_sec", "meeting_url"
    })
    invalid = set(cols) - _ALLOWED_BOT_COLS
    if invalid:
        raise ValueError(f"mark_bot: disallowed column(s): {invalid}")
    sets = ", ".join(f"{k}=?" for k in cols) # column names from the whitelist; values bound via ?
```

Note the values themselves are still bound via `?` placeholders — only whitelist-validated column *names* enter the SQL string. Additionally, all request bodies are parsed through strict Pydantic models before any value reaches the data layer (verified under passed row 5.1.1), and the SQLite database is a private file on the application volume — reachable only through this code path, with no separate network-exposed database service.

Row 39 — 5.1.9 · OS command injection

Tester asked for: evidence that no user-controlled input reaches shell/system commands.

Repository-wide search of every Python source in the deployed service (2026-09-04) for `subprocess` , `os.system` , `os.popen` , `shell=True` , `eval()` , `exec()` returns **zero matches**:

```
> grep -nE "subprocess|os\.system|os\.popen|shell=True|\beval\(|\bexec\(" railway/*.py
(no matches)
```

The service **never spawns a shell or child process**. Audio processing is done in-process (numpy/librosa/torch library calls), outbound integrations use HTTP client libraries, and file I/O uses `pathlib` APIs. With no process-execution call sites, there is no vector for OS command injection.

Row 40 — 5.1.10 · Local/remote file inclusion & path traversal

Tester asked for: demonstration of the stated path sanitization and parent-path rejection.

Control 1 — character-class whitelist on every filesystem-adjacent name. The sanitizer strips everything outside `[A-Za-z0-9_-]` (so `.`, `/`, `\` cannot survive — `../` is structurally impossible in output). Defined at `railway/app.py:97` and applied at **every** point where external data influences a filename:

```
_SPK_RE = re.compile(r"^[A-Za-z0-9_-]+") # app.py:97
spk = _SPK_RE.sub("_", ((d.get("participant") or {}).get("name") or "unknown").strip("_") or "unknown") #
app.py:633 (capture files)
return (_SPK_RE.sub("_", str(v)).strip("_") or "unknown")[:64] # app.py:998, 1088 (Pydantic validators)
f = udir / f"{_SPK_RE.sub('_', speaker).strip('_') or 'unknown'}.npz" # app.py:1053 (enrollment delete)
```

Control 2 — slug whitelist for content routes (verbatim, `railway/app.py:2285, 2297-2299`):

```
GUIDE_SLUGS = ("detect-deepfake-voice-live-call", "ceo-voice-fraud-wire-transfers",
               "deepfake-detector-accuracy")
...
if slug not in GUIDE_SLUGS: # whitelist — never touch the filesystem with user input
    raise HTTPException(status_code=404, detail="no such guide")
```

Control 3 — no remote inclusion: no route loads or includes content from user-supplied URLs; the only user-influenced outbound fetches (calendar iCal, alert webhooks) go through the SSRF allowlist guard already verified under passed row 5.1.5 and are never rendered/executed. All file reads are additionally scoped to the authenticated user's own directory (`_user_capture_dir(user_id)` , `railway/app.py:1001` — IDOR scoping verified under passed row 3.1.4).

Row 41 — 5.2.1 · Malicious file uploads

Tester asked for: evidence of enforced file-type validation and non-executable storage.

The application exposes no file-upload endpoint at all. Repository search (2026-09-04) for `UploadFile` , `File(` , or multipart request *parsing* returns zero upload handlers (the only `multipart` references are in an outbound HTTP client that *sends* data to a third-party API, `railway/app.py:879, 931`).

Audio enters the system exclusively as **base64-encoded raw PCM16 frames over the authenticated WebSocket** (`/api/ws/audio` — token-authenticated and origin-checked, `railway/app.py:537-564`): fixed sample format (16 kHz mono s16le), decoded with `base64.b64decode` and processed as raw bytes (`app.py:634`). There is no filename, no MIME type, and no container format for an attacker to smuggle content through — a "file type" other than raw PCM cannot be expressed on this interface.

Server-side persistence: capture files are written **by the server** with server-generated names (sanitized speaker slug + counter — Row 40) as `.wav` under the private `/data` volume. That directory is **not a static docroot**: nothing in it is ever executed or served as code; audio is only streamed back through authenticated, user-scoped API routes with audio content types. The Python process never executes content from the data volume.

Row 42 — 6.1.1 · No components with known exploitable vulnerabilities

Tester asked for: the actual dependency-scan output covering all dependencies.

`pip-audit` run on 2026-09-04 against the exact production requirements file (`railway/requirements.txt` — the single source of truth; CI installs from this same file):

```
> python -m pip_audit -r railway/requirements.txt --index-url https://pypi.org/simple
No known vulnerabilities found
```

JSON output (same run): **73 packages resolved and audited, 0 packages with vulnerabilities** — i.e., zero known CVEs at any severity, which a fortiori satisfies the CVSS ≥ 7.0 bar. Full audited set:

```
torch==2.14.0, torchaudio==2.11.0, fastapi==0.141.1, uvicorn==0.52.4, pydantic==2.13.5, pydantic-core==2.46.5, numpy==2.2.6,
librosa==0.11.0, soundfile==0.14.0, speechbrain==1.1.1, edge-tts==7.2.8, aiohttp==3.14.3, async-timeout==5.0.1, multidict==6.7.1,
tabulate==0.10.0, typing-extensions==4.16.0, yarl==1.24.5, aiohappyeyeballs==2.7.1, aiosignal==1.4.0, annotated-doc==0.0.5,
annotated-types==0.8.0, attrs==26.1.0, audioread==3.1.0, certifi==2026.7.22, cffi==2.1.1, click==8.5.0, decorator==5.3.1,
```

frozenset==1.8.0, fsspec==2026.7.0, h11==0.16.0, huggingface-hub==1.30.0, hf-xet==1.6.0, httpx==0.28.1, httpcore==1.0.9, filelock==3.32.5, hyperpyyaml==1.2.3, ruamel-yaml==0.18.17, idna==3.19, joblib==1.6.0, cloudpickle==3.1.2, lazy-loader==0.5, msgpack==1.2.2, networkx==3.4.2, numba==0.67.0, llvmlite==0.49.0, pooch==1.9.0, platformdirs==4.11.7, propcache==0.5.2, pyyaml==6.0.3, requests==2.34.2, charset-normalizer==3.5.1, urllib3==2.7.0, ruamel-yaml-clib==0.2.15, scikit-learn==1.7.2, scipy==1.15.3, sentencepiece==0.2.2, soxr==1.1.0, starlette==1.6.0, anyio==4.15.0, exceptiongroup==1.3.1, sympy==1.14.0, mpmath==1.3.0, threadpoolctl==3.6.0, tqdm==4.70.0, typing-inspection==0.4.4, httptools==0.8.0, python-dotenv==1.2.3, watchfiles==1.2.0, websockets==16.1.1, colorama==0.4.6, jinja2==3.1.6, markupsafe==3.0.3, pycparser==3.0

The machine-readable `pip-audit` JSON report is available on request. Dependency freshness is enforced structurally: CI installs from `railway/requirements.txt` (commit `e7d6ac5`), so the audited set and the deployed set cannot drift.

Row 43 — 6.2.1 · Debug modes disabled in production

Tester asked for: production configuration and/or scan results confirming debug functionality is absent.

Configuration (verbatim, `railway/app.py:178-183`):

```
_DEBUG = os.environ.get("SONAVE_DEBUG", "").lower() in ("1", "true", "yes")
app = FastAPI(
    ...
    docs_url="/docs" if _DEBUG else None,
    redoc_url="/redoc" if _DEBUG else None,
    openapi_url="/openapi.json" if _DEBUG else None,
```

`SONAVE_DEBUG` is **unset** in the Railway production environment, so `_DEBUG` is `False`: the interactive docs, ReDoc, and OpenAPI schema do not exist in production, and FastAPI/uvicorn run without debug or auto-reload modes. Unhandled errors return generic JSON (`{"detail": ...}`) with no stack traces, source paths, or environment details (FastAPI production default; no `debug=True` anywhere in the codebase).

Live production verification (2026-09-04):

```
GET https://usesonave.com/docs -> 404
GET https://usesonave.com/redoc -> 404
GET https://usesonave.com/openapi.json -> 404
GET https://usesonave.com/debug -> 404
GET https://usesonave.com/__debug__ -> 404
```

Row 44 — 6.3.1 · Origin header not used for authentication/access control

Tester asked for: direct evidence that authn/z decisions rely on session/bearer tokens and server-side checks rather than the Origin header.

Authentication inputs are exactly three, none of them Origin (`auth.get_principal`, `railway/auth.py` — token sources are the `Authorization: Bearer` header, the `X-Sonave-Token` header, and `HttpOnly` session cookies; the query-string source was deliberately removed, per passed row 2.1.1). Authorization is then decided server-side from the verified session (role/workspace decoded from the HMAC-signed token + DB `session_ver` check — passed rows 3.1.1/3.1.2).

The Origin header appears exactly once in the entire service, and it is a *deny-only* cross-site-WebSocket-hijacking filter applied **after** token authentication has already succeeded — it can only reject, never grant (`railway/app.py:551-563`):

```
# Validate WebSocket Origin to prevent cross-site WebSocket hijacking (CASA AL1 / ASVS V13.5)
_origin = ws.headers.get("origin", "")
_allowed_origins = ("https://usesonave.com", "https://meet.google.com",
                    "https://workspace.google.com", "http://localhost", "http://127.0.0.1")
if _origin and not any(_origin.startswith(o) for o in _allowed_origins):
    logger.warning("ws_audio: rejected disallowed origin %s", _origin)
    await ws.close(code=1008)
    return
```

A request with a forged Origin and no valid token is still rejected (token check at `app.py:537-549` runs first, closing with code 1008); a request with a valid token gains nothing extra from any Origin value. CORS/ `frame-ancestors` are used only for browser-side embedding policy, not authorization.

Row 45 — 6.4.1 · No subdomain takeover

Tester asked for: current DNS records for the domain and subdomains demonstrating no dangling CNAMEs.

Complete zone posture, resolved live 2026-09-04:

```
usesonave.com A 104.21.53.4 (Cloudflare anycast, proxied)
usesonave.com A 172.67.206.143 (Cloudflare anycast, proxied)
usesonave.com AAAA 2606:4700:3036::6815:3504
usesonave.com AAAA 2606:4700:3030::ac43:ce8f
usesonave.com NS leah.ns.cloudflare.com
usesonave.com NS terry.ns.cloudflare.com
usesonave.com MX route1/route2/route3.mx.cloudflare.net (Cloudflare Email Routing)
usesonave.com TXT "v=spf1 include:_spf.mx.cloudflare.net ~all"
usesonave.com TXT google-site-verification=_enkKrnjXWr... (Google ownership proof)
```

- **Zero CNAME records exist anywhere in the zone** — the takeover-prone record type is simply not present. The apex is Cloudflare-proxied A/AAAA to Cloudflare's own anycast edge (not to a claimable third-party hostname).
- Common subdomain probes (`www` , `api` , `app` , `mail` , `staging` , `dev` , `admin`) — **none resolve** (NXDOMAIN). No orphaned records point at decommissioned services; there is nothing to claim.
- The zone is managed solely in the operator's Cloudflare account (registrar + DNS + proxy in one control plane), protected by 2SV; every record above is enumerated, so the subdomain inventory is exactly this list.

Row 46 — 6.5.1 · No credentials/payment details in logs; tokens never logged in reversible form

Tester asked for: logging controls and representative login-process log samples.

Masking control (verbatim, `railway/app.py:124-128`) and both login call sites (`app.py:2034` , `2068`):

```
def _mask_email(email: str | None) -> str:
    """d***@x.com - logs identify without exposing (ASVS 6.5.1 over-compliance)."""
    ...
logger.info("login: %s (%s)", _mask_email(user.get("email")), user.get("role"))
logger.info("credential login: %s (%s)", _mask_email(user.get("email")), user.get("role"))
```

Real captured log sample (login flow exercised via the automated test suite on 2026-09-04 — identical code path to production logging):

```
INFO sonave.capture:app.py:2068 credential login: g***@example.com (member)
```

- **Credentials:** the application never possesses passwords (Google OAuth — passed rows 1.1.2/1.1.3), so none can be logged.
- **Session tokens:** no `logger` call in the codebase includes a session, bearer, bot, or API token. Where tokens must be persisted for lookup they are stored **only as SHA-256 digests** (irreversible — `railway/app.py:372` , `railway/db.py:604`); the same digests are the only token-derived values that could ever appear in diagnostics.
- **Payment details:** card data never touches Sonave servers — checkout and card entry happen on Stripe-hosted pages; Sonave receives only opaque customer/subscription IDs via signature-verified webhooks (`railway/billing.py:61-72`).
- Log access is limited to the Railway project's sole member (2SV-protected — Row 48).

Row 47 — 6.6.1 · Browser storage securely cleared on logout

Tester asked for: identification of browser-stored authentication data and proof it is cleared on logout.

Inventory of browser-stored authentication material (complete):

Surface	Storage	Cleared on logout by
Web console (usesonave.com)	sonave_token / session cookies — HttpOnly, Secure (never in localStorage; JS cannot read them)	Server-side delete_cookie on POST /auth/logout
Meet add-on (partitioned iframe, cookie-less by necessity)	session token in localStorage / sessionStorage (sonave_tok)	clearTok() in the signout handler, before the logout request

Server-side cookie clearing (verbatim, railway/app.py:2099-2103):

```
resp.delete_cookie("sonave_token", path="/") # HttpOnly - JS can't clear it
resp.delete_cookie(auth.SESSION_COOKIE, path="/")
resp.headers.append("set-cookie",
                    f"{auth.PARTITIONED_COOKIE}=; Max-Age=0; Path=/; Secure; HttpOnly; "
                    f"SameSite=None; Partitioned")
```

Add-on storage clearing (verbatim, railway/meet-addon.html:232 invoked at :281 in signout()):

```
function clearTok(){TOK='';try{localStorage.removeItem('sonave_tok');sessionStorage.removeItem('sonave_tok')}}catch(x){}}
```

The signout flow then sets the signed-out flag, disables Google auto-select, and calls POST /auth/logout . **Defense in depth:** logout also increments the server-side session_ver and revokes the Google OAuth token (passed row 2.2.1) — so even a byte-for-byte copy of any cleared client value is dead server-side the moment logout completes. No other confidential data is kept in browser storage (remaining keys are UI preferences: language, onboarding flags).

Row 48 — 6.7.1 · Secure storage of access tokens, API keys, server-side secrets

Tester asked for: the documented secrets access-control policy, storage evidence, and logging/monitoring of access.

Policy document: docs/secrets-policy.md (submitted alongside this pack) — covers inventory, storage, access control, rotation, revocation, and monitoring.

Storage & injection: all secrets (Google OAuth client secret, session HMAC secret, SONAVE_API_TOKEN , Stripe keys, Recall/Modal/HF tokens) exist only as **Railway environment variables**, injected at process start; Railway stores service variables encrypted at rest. Repository search confirms zero hardcoded secrets — every secret is read via os.environ.get(...) (e.g. railway/app.py:95 , railway/auth.py), .env is git-ignored, and the public repo history contains no credential material. CI secrets live in GitHub Actions encrypted secrets, referenced only as \${ secrets.* } (.github/workflows/ci-cd.yml:57-60). User OAuth refresh tokens are stored in the application database on Railway's encrypted persistent volume, accessible only through the authenticated application (rows 3.1.x) — never returned by any API.

Access control: the Railway project, Cloudflare account, GitHub repository, and Google Cloud console each have **exactly one member** (the operator), all protected by 2-Step Verification (FIDO2 — passed row 3.3.1). There are no shared accounts and no additional principals to restrict; least privilege is the degenerate single-operator case.

Logging & monitoring: Railway records dashboard/config access and deployment events in the project activity log (operator-reviewable); every application-level authentication and session event is tracked in the audit events table and pushed to the operator in real time (admin dashboard + email/Slack notification pipeline). Exposure response is documented in the policy: rotate in Railway → redeploy → bump_session_ver global invalidation where applicable.

Prepared for upload against ESOF rows 29, 30, 34, 37–48. All live outputs captured 2026-09-04 from the production deployment at https://usesonave.com. Related: docs/casa-addendum.md (remediation ledger for controls hardened during the assessment window), docs/secrets-policy.md .

Sonave — Secrets Management Policy

Owner: Derek Gallardo (sole operator) · **Effective:** 2026-09-04 · **Review:** quarterly, and immediately after any suspected exposure.

1. Inventory

Secret	Purpose	Storage location
SONAVE_SESSION_SECRET	HMAC-SHA256 session signing key (256-bit)	Railway env var
SONAVE_API_TOKEN	Operator/machine token	Railway env var
SONAVE_GOOGLE_CLIENT_SECRET	Google OAuth client secret	Railway env var
Stripe secret + webhook signing keys	Billing	Railway env var
Recall.ai / Modal / Hugging Face tokens	Bot deploy, GPU scorer, model hub	Railway env vars; CI copies in GitHub Actions encrypted secrets
User Google OAuth refresh tokens	Calendar auto-join (flag-gated)	Application DB on Railway encrypted persistent volume
Per-bot stream tokens, user API keys	Audio ingest, API access	SHA-256 digest only in DB; plaintext shown once at creation, never stored

2. Storage rules

- Secrets exist **only** in Railway environment variables (encrypted at rest by the platform) or GitHub Actions encrypted secrets. Never in source code, git history, container images, logs, or client-side code. `.env` (local dev) is git-ignored.
- Application code reads secrets exclusively via `os.environ.get(...)` at startup.
- Derived credentials (bot tokens, API keys) are persisted only as irreversible SHA-256 hashes.

3. Access control

- One human principal (the operator) on each control plane: Railway project, GitHub repository, Cloudflare account, Google Cloud console, Stripe account. No shared or service accounts with dashboard access.
- Every operator account enforces 2-Step Verification (FIDO2 security key / passkey).
- Production runtime is the only non-human reader; CI receives only the deploy-scoped tokens it needs (`MODAL_TOKEN_*` , `SONAVE_API_TOKEN` , `HF_TOKEN`) via `${{ secrets.* }}` references.

4. Rotation & revocation

- Routine rotation:** third-party keys (Stripe, Recall, Modal, HF, Google client secret) are rotated via their consoles and updated in Railway; the service picks them up on redeploy.
- Session signing secret:** rotating `SONAVE_SESSION_SECRET` invalidates every outstanding session token at once (accepted behavior — users re-authenticate via Google).
- Targeted revocation:** per-user `session_ver` bump invalidates all of one user's sessions immediately; Google OAuth tokens are revoked at `oauth2.googleapis.com/revoke` on logout or disconnect; user API keys and bot tokens are deleted by row (hash-stored, so deletion is final).
- On suspected exposure (in order, same day):** rotate the secret at its source → update Railway env var → redeploy → global `session_ver` bump if session-adjacent → review Railway/Cloudflare activity logs and the application audit trail for misuse → record the incident and remediation in `docs/casa-addendum.md`.

5. Monitoring

- Railway project activity log records configuration and deployment access (operator-reviewed).
- The application audit trail (`events` table) records every sign-in, sign-out, session revocation, key creation/deletion, and bot lifecycle event, and pushes each to the operator in real time (admin dashboard + email/Slack).
- CI is the only automated writer to production and installs from the single pinned requirements source; dependency health is checked with `pip-audit`.